

Predicting California Housing Values with Ridge Regression and PyTorch Neural Networks

Ruo Chen Li

1 Introduction

The prediction of the housing value is a useful regression problem as it depends on multiple socioeconomic and geographic factors. In this project, I study whether median housing values in California can be predicted from census data and what method is the best in making such prediction. The main research question is Can a PyTorch neural network improve California median house value prediction compared with Ridge Regression, and how important are geographic features such as latitude and longitude for predictive performance? A California Housing dataset is used for this research question. Ridge Regression is used as a regularized linear baseline model, while a feedforward neural network is used to test whether the nonlinear relationships among income, housing age, population, occupancy, and location improve prediction accuracy. I also study on whether latitude and longitude help improve the prediction.

2 Dataset

The data set used for this research is the California Housing Dataset from scikit-learn. It contains 20,640 samples from California census data. The target variable is the median house value. The input features include median income, house age, average number of rooms, average number of bedrooms, population, average occupancy, latitude, and longitude.

The feature set was studied in two versions. The first version used all eight feature, while the second version excluded the geographical information, latitude and longitude. This comparison was used to test whether geographic location is important for prediction.

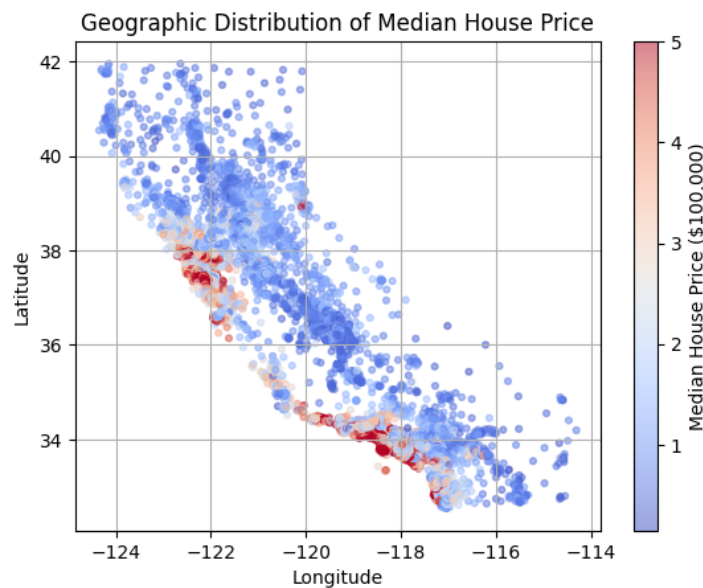


Figure 1: Geographic distribution of housing values in the California Housing dataset. Each point represents a census block group, and color indicates median house value. The spatial pattern suggests that latitude and longitude may provide useful predictive information.

3 Methods

3.1 Ridge Regression

Ridge Regression was used as the linear baseline model. Ridge Regression minimizes the squared error while adding a L_2 penalty on the coefficients of the model:

$$\min_{\theta} \sum_{i=1}^n (y_i - x_i^T \theta)^2 + \alpha \|\theta\|^2$$

α is the regularization parameter that controls the strength of the penalty. All input features were standardized before fitting the model, as the Ridge Regression is sensitive to feature scale. A 5-fold cross-validation study was used to select α from candidate values. The selected value was then used to train the final model on the full training set and evaluate it on testing set.

3.2 PyTorch Neural Network

The nonlinear model was a feedforward neural network implemented in PyTorch. The model used two hidden layers with ReLU activation functions and a single scalar output for the predicted median house value. Input features were standardized before training. The neural network architecture was selected using 5-fold cross-validation over four candidate hidden layer sizes: (16,8), (32,16), (64,32) and (128,64). These architectures were tested to capture nonlinear relationships while limiting overfitting. The architecture with the lowest mean validation MSE was selected and then retrained on the full training set.

The result of cross validation is shown in figure 2:

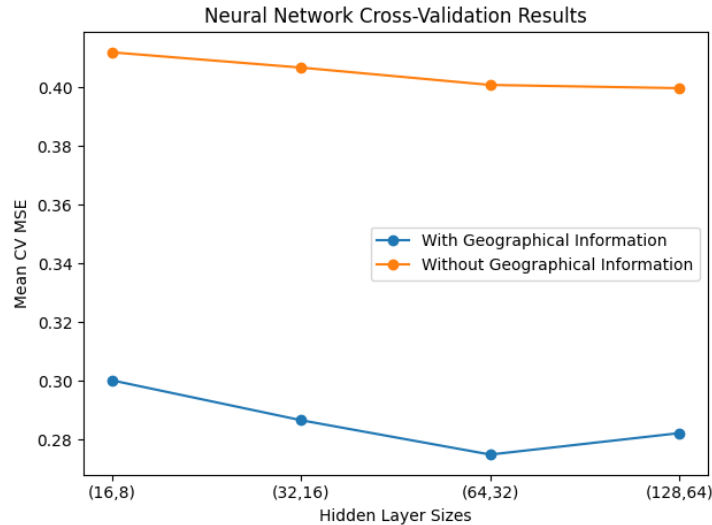


Figure 2: Neural network cross-validation results for two feature sets. The all-feature model achieved lower CV MSE than the model without latitude and longitude across all candidate architectures. With geographical information, the architecture (64, 32) achieved the lowest mean CV MSE and was selected for final testing. Without geographical information, (128,64) has achieved the lowest mean CV MSE and was selected.

3.3 Impact of Geographical Features

To evaluate the importance of geographic features, both Ridge and Regression and the neural network were trained in two settings: one use all features and one excluding latitude and longitude. This study tests whether location information improves predictive performance, and whether the nonlinear neural network can make better use of geographic information than the linear Ridge model.

4 Results

Table 1 summarizes the cross-validation and testing set result for all four experiments. The neural network with all features achieved the best performance, with a test MSE of 0.2686, and the test R^2 of 0.7950. In comparison, Ridge Regression with all features achieved a test MSE of 0.5559, and the test R^2 of 0.6126. This indicates the neural network substantially outperformed Ridge Regression.

Table 1: Model comparison results. MSE values are reported in squared units of \$100,000, while R^2 is dimensionless.

Model	Feature Set	Selected Parameter	CV MSE	Test MSE	Test R^2
Ridge	All Features	$\alpha = 0.01$	0.5193	0.5559	0.6126
Ridge	No Geographical	$\alpha = 10$	0.6116	0.6417	0.5103
Neural Network	All Features	(64,32)	0.2772	0.2686	0.7950
Neural Network	No Geographical	(128,64)	0.4014	0.4271	0.6741

Compare between different feature sets, using both Ridge Regression and neural network, the set with all features outperform the set without the geographical features, achieving lower test MSE and higher test R^2 .

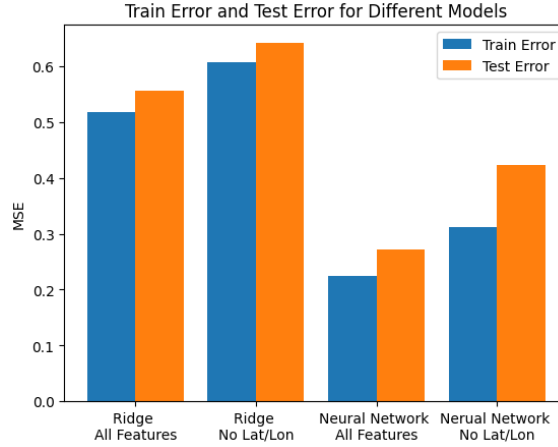


Figure 3: Train and test MSE comparison across Ridge Regression and neural network models. Lower MSE indicates better prediction performance on the held-out train and test set. Neural network with all features has the lowest MSE, and Ridge Regression without geographical features has the highest MSE

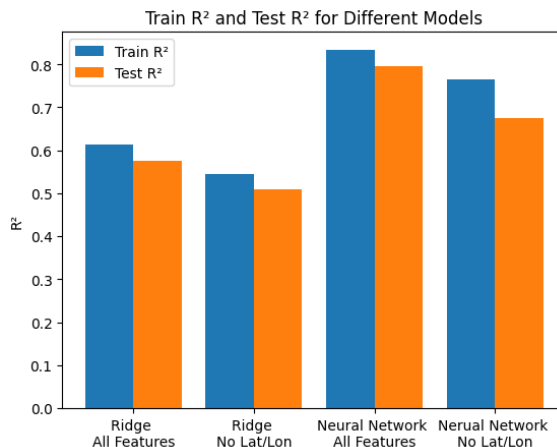


Figure 4: Train and test R^2 comparison across Ridge Regression and neural network models. Higher R^2 indicates better prediction performance on the held-out train and test set. Neural network with all features has the highest R^2 , and Ridge Regression without geographical features has the lowest R^2 .

5 Conclusion

This project compared Ridge Regression and neural network for predicting California median house values. The results show that the neural network performed much better than the linear baseline. The best neural network reached test R^2 of 0.795, which indicates that it can explain about 79.5% of the variation in test set housing values, while the best linear baseline model can only explain 61.26% of the test set values.

The study of geographical features showed that the latitude and longitude data are critical in predicting the median housing price. For both Ridge Regression models and neural network models, the performance of the models trained without geographical features had worse performance to those which use all features. This indicates that the housing median housing prices in California have strong spatial patterns.

There are also some limitations. First, due to the scale of this project, the hyperparameter searches for both neural network and Ridge Regression is limited to few test points. There are only 2 hidden layers for the neural network, which could disable it from capture more complicated nonlinear relationship. Future work could include additional model for comparison or perform more complicated data regularization and selections.

6 Sources

The dataset used in this project was the California Housing dataset provided through scikit-learn. The machine learning implementation used scikit-learn for Ridge Regression, preprocessing, cross-validation, train/test splitting, and evaluation metrics. PyTorch was used to implement and train the neural network model. NumPy, pandas, and Matplotlib were used for numerical computation, data manipulation, and visualization.

A Code Appendix

This section contains critical code used in this project. For other code, please visit this Github Repository

A.1 Neural Network

```
1 import numpy as np
2 import pandas as pd
3
4 import torch
5 import torch.nn as nn
6 from torch.utils.data import TensorDataset, DataLoader
7
8 from sklearn.model_selection import train_test_split, KFold
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
11
12 df= pd.read_csv("california_housing.csv")
13 ## without Geographical info
14 X = df.drop(columns = ["Latitude", "Longitude", "MedHouseVal"])
15 ## with Geographical info
16 #X = df.drop(columns = ["MedHouseVal"])
17 y = df["MedHouseVal"]
18
19 X_train, X_test, y_train, y_test = train_test_split(
20     X, y,
21     test_size=0.2,
22     random_state=42
23 )
24
25 class HousingNN(nn.Module):
26     def __init__(self, input_dim, hidden1=64, hidden2=32):
27         super().__init__()
28         self.net = nn.Sequential(
29             nn.Linear(input_dim, hidden1),
30             nn.ReLU(),
31             nn.Linear(hidden1, hidden2),
32             nn.ReLU(),
33             nn.Linear(hidden2, 1)
34         )
35
36     def forward(self, x):
37         return self.net(x)
38
39 def train_nn(X_train_np, y_train_np, X_val_np, y_val_np,
40             hidden1 = 64, hidden2 = 32,
41             lr = 0.001, epochs = 100, batch_size = 64):
42     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
43
44     X_train_tensor = torch.tensor(X_train_np, dtype=torch.float32)
45     y_train_tensor = torch.tensor(y_train_np, dtype=torch.float32).view(-1, 1)
46
47     X_val_tensor = torch.tensor(X_val_np, dtype=torch.float32).to(device)
48
49     train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
50     train_loader = DataLoader(train_dataset, batch_size = batch_size, shuffle=True)
51
52     model = HousingNN(
53         input_dim=X_train_np.shape[1],
54         hidden1=hidden1,
55         hidden2=hidden2
56     ).to(device)
57
58     loss_fn = nn.MSELoss()
```

```

59     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
60
61     for epoch in range(epochs):
62         model.train()
63
64         for batch_X, batch_y in train_loader:
65             batch_X = batch_X.to(device)
66             batch_y = batch_y.to(device)
67
68             y_pred = model(batch_X)
69             loss = loss_fn(y_pred, batch_y)
70             optimizer.zero_grad()
71             loss.backward()
72             optimizer.step()
73
74         model.eval()
75         with torch.no_grad():
76             val_pred = model(X_val_tensor).cpu().numpy().reshape(-1)
77
78         val_mse = mean_squared_error(y_val_np, val_pred)
79
80         return model, val_mse
81
82 hidden_size_options = [
83     (16, 8),
84     (32, 16),
85     (64, 32),
86     (128, 64),
87 ]
88
89 kf = KFold(n_splits=5, shuffle=True, random_state=42)
90
91 cv_results = []
92 X_train_df = X_train.reset_index(drop=True)
93 y_train_series = y_train.reset_index(drop=True)
94
95 for hidden1, hidden2 in hidden_size_options:
96     fold_mses = []
97
98     for train_idx, val_idx in kf.split(X_train_df):
99         X_fold_train = X_train_df.iloc[train_idx]
100         X_fold_val = X_train_df.iloc[val_idx]
101         y_fold_train = y_train_series.iloc[train_idx]
102         y_fold_val = y_train_series.iloc[val_idx]
103
104         scalar = StandardScaler()
105         X_fold_train_scaled = scalar.fit_transform(X_fold_train)
106         X_fold_val_scaled = scalar.transform(X_fold_val)
107
108         _, val_mse = train_nn(
109             X_fold_train_scaled,
110             y_fold_train.to_numpy(),
111             X_fold_val_scaled,
112             y_fold_val.to_numpy(),
113             hidden1=hidden1,
114             hidden2=hidden2,
115             lr=0.001,
116             epochs=100,
117             batch_size=64
118         )
119
120         fold_mses.append(val_mse)
121
122
123 mean_cv_mse = np.mean(fold_mses)

```

```

124
125     cv_results.append({
126         "hidden1": hidden1,
127         "hidden2": hidden2,
128         "CV MSE": mean_cv_mse
129     })
130
131     print(f"Hidden sizes ({hidden1}, {hidden2}) - CV MSE: {mean_cv_mse:.4f}")
132
133 cv_results_df = pd.DataFrame(cv_results)
134 print("\nCross-validation results:")
135 print(cv_results_df)
136
137 best_row = cv_results_df.loc[cv_results_df["CV MSE"].idxmin()]
138 best_hidden1 = int(best_row["hidden1"])
139 best_hidden2 = int(best_row["hidden2"])
140
141 print("\nBest hidden sizes:", (best_hidden1, best_hidden2))
142
143 final_scaler = StandardScaler()
144 X_train_scaled = final_scaler.fit_transform(X_train)
145 X_test_scaled = final_scaler.transform(X_test)
146
147 final_model, _ = train_nn(
148     X_train_scaled,
149     y_train.to_numpy(),
150     X_test_scaled,
151     y_test.to_numpy(),
152     hidden1=best_hidden1,
153     hidden2=best_hidden2,
154     lr=0.001,
155     epochs=200,
156     batch_size=64
157 )
158
159
160 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
161
162 final_model.eval()
163 with torch.no_grad():
164     X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
165     y_pred = final_model(X_test_tensor).cpu().numpy().reshape(-1)
166
167 print("\nFinal PyTorch NN Test Results:")
168 print("Test MSE:", mean_squared_error(y_test, y_pred))
169 print("Test R2:", r2_score(y_test, y_pred))

```

A.2 Ridge Regression

```

1 import sklearn
2 import numpy as np
3 import pandas as pd
4 from sklearn.linear_model import Ridge
5 from sklearn.pipeline import make_pipeline
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.model_selection import train_test_split, GridSearchCV
8 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
9
10
11 df = pd.read_csv("california_housing.csv")
12
13 ## Without Geographical Info
14 #X=df.drop(columns = ["MedHouseVal", "Latitude", "Longitude"])
15

```

```

16 ## With Geographical Info
17 X=df.drop(columns = ["MedHouseVal"])
18 y=df["MedHouseVal"]
19
20 X_train, X_test, y_train, y_test = train_test_split(X,y,test_size = 0.2, random_state=42)
21
22 ridge_model = make_pipeline(
23     StandardScaler(),
24     Ridge()
25 )
26
27 parameter_grid = {
28     "ridge__alpha": [0.01, 0.1, 1, 10, 100]
29 }
30
31 ridge_cv = GridSearchCV(
32     ridge_model,
33     parameter_grid,
34     cv = 5,
35     scoring = "neg_mean_squared_error"
36 )
37
38 ridge_cv.fit(X_train, y_train)
39
40 print("Best alpha:", ridge_cv.best_params_)
41 print("Best CV MSE:", -ridge_cv.best_score_)
42
43 y_pred = ridge_cv.predict(X_test)
44
45 print("Test MSE:", mean_squared_error(y_test, y_pred))
46 print("Test R2:", r2_score(y_test, y_pred))

```